

Multi-Agent Business Research Assistant

AI Prompts Used

Take-Home Assignment Submission

Synapse AI Solutions

Isra Chaudry

1. Overview

This document lists all AI prompts used during development of the Multi-Agent Business Research Assistant, along with the reasoning behind each. The assignment permitted use of AI tools, so this document is included as required by the submission guidelines.

The project was built using:

- LangGraph — for the 4-agent orchestration and conditional routing
- Groq API (llama-3.3-70b-versatile) LLM provider via the groq Python SDK
- Tavily — for real-time company search
- FastAPI — backend REST API with in-memory session storage
- React + TypeScript (Vite) frontend chat interface

AI tools used:

- ChatGPT, used personally to convert the assignment brief into a project specification
- Claude Code ran autonomously from that specification to build the full codebase

2. System Architecture

The system is directed at LangGraph workflow. Each agent has a single responsibility and routes to the next agent based on its output.

2.1 Agent Flow

START

|

v

Clarity Agent

|-- query is clear -----> Research Agent

|-- query is ambiguous -----> Return clarification question to user
(user replies, workflow continues)

v

Research Agent

|-- confidence score ≥ 6 -----> Synthesis Agent

|-- confidence score < 6 -----> Validator Agent

v

Validator Agent

```
-- result sufficient -----> Synthesis Agent
-- result insufficient, attempts < 3 -----> loop back to Research Agent
-- attempts >= 3 -----> Synthesis Agent (force proceed)
```

v

Synthesis Agent

|

v

END

2.2 Agent Descriptions

Clarity Agent

- Checks whether the query contains enough information to research
- Resolves pronouns like 'their' or 'the CEO' using conversation history
- Outputs: clarity_status, company_name, clarified_query, clarification_question

Research Agent

- Calls Tavily to search for real company data news, financials, leadership, competitors
- Deduplicates results and summarises findings using Groq
- Outputs: research_findings, sources, confidence_score (0 to 10)

Validator Agent

- Assesses whether research findings are complete enough to answer the query
- Decide whether to send Research Agent for another attempt (max 3 total)
- Outputs: validation_result, validation_notes, attempts

Synthesis Agent

- Writes the final structured response using research findings and conversation history
- Formats the response with sections, bullets, and source references
- Outputs: final_answer

3. My Prompt

The following is the only prompt I personally wrote. I gave this to ChatGPT, which produced a detailed CLAUDE.md instruction file. I then gave that file to Claude Code, which used it to autonomously generate the entire project.

Prompt —: My Prompt (Given to ChatGPT)

Prompt used:

"I have a take-home assignment to build a multi-agent research assistant using LangGraph. The system needs 4 specialized agents: a Clarity Agent that checks if a query is specific enough, a Research Agent that searches for company data using Tavily, a Validator Agent that checks research quality, and a Synthesis Agent that writes the final response. It also needs human-in-the-loop clarification, multi-turn conversation memory, conditional routing between agents, a FastAPI backend, and a simple React frontend. I am using Groq as the LLM provider and Tavily for search. Can you generate a detailed Claude Code instruction file (CLAUDE.md) I can use to build this project?"

Reasoning:

I didn't want to jump straight into building without understanding what the assignment was asking. There were a few terms thrown around like LangGraph, conditional routing, and human-in-the-loop that I wanted to understand correctly before writing any code. I gave ChatGPT the full assignment requirements and asked it to turn them into a structured spec file tailored to my tech stack. The resulting CLAUDE.md was then handed to Claude Code which built the whole project from it.

4. Claude Code Prompts

The following prompts were not written by me. They were generated and used autonomously by Claude Code as it worked through the project from the CLAUDE.md spec. They are listed here for full transparency as required by the assignment.

Prompt 1: Architecture Planning

Prompt used:

"Help me design a LangGraph multi-agent research assistant with clarity, research, validator, and synthesis agents. The system should support human-in-the-loop clarification, multi-turn conversation memory, and conditional routing between agents."

Reasoning:

Used to plan the overall system architecture, define the responsibilities of each agent, and decide how LangGraph conditional edges should be structured to avoid over-engineering while meeting all assignment requirements.

Prompt 2: Groq LLM Integration

Prompt used:

"Show how to call the Groq API using the groq Python SDK for a multi-agent LangGraph system. Include JSON output mode and safe fallback parsing."

Reasoning:

Used to implement llm_service.py using the Groq SDK directly. Needed a reliable JSON parsing fallback because LLaMA models sometimes wrap responses in markdown even when instructed not to — having it handled in one place meant no try/catch blocks inside every agent.

Prompt 3: LangGraph Routing**Prompt used:**

"Show how to implement conditional routing in LangGraph for clarity, research confidence, and validation loops. Include route-after functions and StateGraph setup."

Reasoning:

The interrupt and resume flow was the most complex part of the backend. Used to correctly implement workflow.py with add_conditional_edges, entry point configuration, and the research retry loop up to 3 attempts.

Prompt 4: Clarity Agent Design**Prompt used:**

"Design a clarity agent that checks if a business research query is specific enough. It should resolve pronouns using conversation history and session context and return a structured JSON response."

Reasoning:

Used to write the system prompt for the Clarity Agent and define the JSON output schema including clarity_status, clarification_question, company_name, and clarified_query.

Prompt 5: Research Agent with Tavily**Prompt used:**

"Implement a research agent that uses Tavily to search for company data, deduplicates results, and uses an LLM to summarize findings and produce a confidence score."

Reasoning:

Used to structure the Research Agent multi-query approach, deduplication logic, and confidence scoring prompt that feeds the routing decision.

Prompt 6: FastAPI Session Memory**Prompt used:**

"Implement a FastAPI /chat endpoint that maintains per-session conversation history and handles a pending clarification flow where the user's next message is combined with the original pending query."

Reasoning:

Used to design the session store structure and the clarification resolution logic in main.py, including how to merge pending queries with clarification answers.

Prompt 7: React Chat UI, Initial Build**Prompt used:**

"Create a React TypeScript chat interface using Vite with message bubbles, a loading indicator, source links as clickable URLs, and clarification question styling. Use a dark-themed CSS layout with a fixed input bar at the bottom. Each session should generate a UUID as the session_id sent to the FastAPI backend."

Reasoning:

Used to scaffold the initial frontend with App.tsx, ChatWindow.tsx, MessageBubble.tsx, and api/chat.ts. Established the component structure, session UUID pattern, and API fetch logic.

Prompt 8: Clarification Loop Bug Fix**Prompt used:**

"The clarity agent keeps asking for clarification even after the user has already answered it. Fix the clarification resolution flow so that when a user answers a clarification question, the pending query is combined cleanly with the answer and the clarity agent is bypassed on the next run."

Reasoning:

Used to fix two bugs: the Clarity Agent was over-triggering on queries where a name was already present, and answering a clarification caused an infinite loop. Fixed by pre-setting clarity_status to clear and company_name in the LangGraph state when resolving a clarification.

Prompt 9: Frontend UI Redesign**Prompt used:**

"Redesign the React frontend to look like a professional AI assistant dashboard similar to Claude or ChatGPT. Add a collapsible sidebar with conversation history. Support multiple independent chat sessions. Add proper markdown rendering using react-markdown and remark-gfm. Add light and dark mode. Style assistant messages with an avatar and show sources as a card below the response."

Reasoning:

The initial frontend was functional but looked like a prototype. The markdown rendering was also a real need since the Synthesis Agent returns structured markdown and without react-markdown it was showing raw asterisks and hashes in the chat window.

Prompt 10: Color Scheme**Prompt used:**

"Change the color scheme to use a warm cream background for light mode and near-black charcoal for dark mode. Use burgundy as the accent color for buttons, user message bubbles, and highlights. Make it look like a senior frontend UI designer made it."

Reasoning:

The initial teal accent felt generic. A burgundy and cream palette felt more intentional and premium. Using CSS custom properties meant switching between light and dark mode was clean without duplicating style rules.

Prompt 11: *Sidebar Styling*

Prompt used:

"Change the sidebar background to match the cream color of the main area. Keep burgundy as the accent for active items and the new conversation button. Add a border on the right edge of the sidebar to visually separate it from the chat area."

Reasoning:

The dark sidebar was creating too much visual contrast against the cream main area and made it look like two separate apps. The right-side border gives the needed separation without making the sidebar feel heavy.

Prompt 12: *Centered Input on Empty State*

Prompt used:

"When there are no messages, move the chat input bar from the bottom of the screen to be centered on the page, grouped with the welcome text and example chips, like Claude.ai landing state. When a conversation starts the input moves back to the fixed bottom footer."

Reasoning:

A fixed empty chat window with a bottom input bar looked unfinished. The centered empty state with example prompts makes it immediately clear what the app does and gives the demo a much cleaner starting point.

Prompt 13: *Debugging and Completeness Review*

Prompt used:

"Review this multi-agent LangGraph backend for missing requirements, routing edge cases, and integration issues between the session store, graph state, and FastAPI endpoint."

Reasoning:

I didn't want something breaking during the demo that I hadn't thought about. Things like Tavily returning nothing, the validator looping forever, or the Groq client failing at import time are exactly the kind of stuff that goes wrong at the worst moment. This caught the history slice ordering in main.py, the attempts counter initialization, and lazy client initialization for Groq and Tavily.
